

Degree Project in Media Technology

First cycle, 15 credits

Machine Learning for Housing Price Prediction in Stockholm's Inner City

A Comparative Analysis

ALEXANDER MALEKI

ANDREA WOLDEMAR

TRITA – XXX-XXX 20XX:XX
Stockholm, Sweden 2025

www.kth.se

Machine Learning for Housing Price Prediction in Stockholm's Inner City: A Comparative Analysis

Alexander Maleki
KTH Royal Institute of Technology,
School of Electrical Engineering and Computer Science
alex9@kth.se

Andrea Woldemar
KTH Royal Institute of Technology,
School of Electrical Engineering and Computer Science
woldemar@kth.se

Abstract

Predicting housing prices is a complex task influenced by numerous property-specific and locational factors, which can significantly impact market dynamics. To address this challenge, the work presented in this thesis evaluates and compares the predictive performance of three supervised machine learning models—Random Forest, Gradient Boosting (XGBoost), and K-Nearest Neighbors (KNN)—for apartment prices in Stockholm's inner city. Real estate transaction data was collected exclusively from Booli, a public platform for housing market information in Sweden. The models were trained on historical sales data and evaluated on recent transactions using a time-based train-test split approach. Standard regression metrics, including Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE), were used to assess model performance. The results demonstrate that tree-based models, particularly XGBoost, achieve quite accurate predictive performance compared to KNN. The findings offer insights for real estate investors, buyers, and market analysts, while contributing to the growing research on machine learning applications in the real estate domain.

Keywords

Machine Learning, Housing Price Prediction, Real Estate Market, Supervised Learning, Random Forest, Gradient Boosting, XGBoost, K-Nearest Neighbors, Model Evaluation, Stockholm Housing Market, Predictive Analytics

Sammanfattning

Att förutsäga bostadspriser är en utmanande uppgift som påverkas av många bostadsspecifika och geografiska faktorer, vilka kan ha en betydande påverkan på marknadsdynamiken. För att hantera denna utmaning utvärderar och jämför det arbete som presenteras i detta examensarbete den prediktiva förmågan hos tre övervakade maskininlärningsmodeller—Random Forest, Gradient Boosting (XGBoost) och K-Nearest Neighbors (KNN)—för lägenhetspriser i Stockholms innerstad. Fastighetstransaktionsdata samlades in uteslutande från Booli, en offentlig plattform för bostadsinformation i Sverige. Modellerna tränades på historiska försäljningsdata och utvärderades på nyare transaktioner genom en kronologisk uppdelning av tränings- och testdata. Standardmetoder för regressionsanalys, inklusive Root Mean Squared Error (RMSE) och Mean Absolute Error (MAE), användes för att mäta modellernas prestanda. Resultaten visar att trädbaserade modeller, särskilt XGBoost, når relativt hög prediktiv noggrannhet jämfört med KNN. Slutsatserna ger värdefulla insikter för fastighetsinvesterare, köpare och marknadsanalytiker och bidrar till den växande forskningen om maskininlärningens tillämpningar inom fastighetssektorn.

1 Introduction

Housing is one of the most important aspects of people's everyday lives and one of the largest financial decisions individuals make. In large urban areas such as Stockholm, housing prices have risen sharply over the past decades, creating challenges for both homebuyers and policymakers [21]. Predicting housing prices accurately is therefore of great importance—not only for individuals buying or selling property but also for banks, real estate agents, urban planners, and economists who rely on these forecasts for planning and decision-making.

Housing prices are influenced by a variety of factors, such as the size and condition of the apartment, its location, economic conditions, and even the time of year. Given the complexity of these relationships, machine learning has become an increasingly important tool in recent years for predicting housing prices, as demonstrated in several studies [20, 30, 32]. Unlike traditional statistical models, machine learning methods can capture non-linear patterns and process large datasets with many features, making them well-suited for this task [15, 24].

This thesis focuses on the application of three supervised machine learning models to predict apartment prices in Stockholm's inner city: Random Forest, Gradient Boosting (specifically XGBoost), and K-Nearest Neighbors. The aim is to compare how well these models perform when applied to real housing market data and to evaluate whether tuning their hyperparameters can lead to better predictions. The study uses historical apartment sales data from Booli and applies time-aware validation methods to simulate real-world prediction conditions.

By systematically evaluating different modeling approaches on real-world sales data, this thesis aims to provide valuable insights for investors, homebuyers, and researchers interested in the application of machine learning techniques within housing markets.

2 Background

This section aims to give an overview of the key concepts in this study. First, an introduction to Stockholm's real estate market is presented, providing insight regarding historical events, economic influences, and other factors affecting housing prices. Moreover, the models used in this research are presented—Random Forest, Gradient Boosting, and K-Nearest Neighbors. Their methodologies, strengths, relevance to predictive analysis, and the importance of hyperparameter tuning for model performance are explained.

Furthermore, this section outlines the dataset used for model training and evaluation, providing detailed information on its features and preparation. The geographical scope of the study is then defined, focusing on Stockholm's inner city. The section concludes by presenting related work, summarizing prior research in the field, and identifying gaps that this study aims to address.

2.1 The Real Estate Market in Stockholm

Stockholm's housing market has experienced many changes over the years, shaped by economic trends, government policies, and supply and demand. Housing prices have followed cycles of rapid increases and declines, often influenced by credit availability, population growth, and financial speculation.

Historically, real estate in Stockholm has been a valuable asset, with price trends dating back to at least the 18th century. Ingman [21] developed a long-term housing price index covering 1730 to 2020, showing that recent price surges are not unique. A similar surge occurred in the late 1800s, followed by a crash in 1887 that led to an economic downturn [21]. More recently, from 1993 to 2020, small houses in Stockholm increased in price by 340%, while apartment prices rose by 430%.

Several factors influencing housing prices in Stockholm have been identified by Ingman [21]:

- **Economic Cycles**: Periods of financial expansion have led to rising prices, while economic downturns have resulted in price drops. For example, financial deregulation in the 1980s made it easier to access credit, which led to rising home values. However, the 1990s banking crisis triggered a sharp decline in prices.
- **Population Growth and Urbanization**: Stockholm's population has steadily increased, creating high demand for housing. The industrialization of the late 19th and early 20th centuries significantly expanded the city, and even today, demand continues to outpace supply.
- **Limited Housing Supply**: High prices are driven by strict zoning laws and lack of available land. Despite some industrial areas being redeveloped for housing, new construction has been insufficient to meet the growing demand.
- **Financialization of Real Estate**: Since the 1980s, housing has become an important investment asset, with rising mortgage-backed investments leading to increased speculation and price fluctuations.

In Stockholm, a large share of apartments are classified as tenant-owned housing (*bostadsrätter*, Swedish for tenant-owned apartments), a form of cooperative ownership where residents own the right to occupy a unit and pay a monthly fee to the housing association. These fees cover shared expenses such as maintenance and utilities, and therefore represent an important component of total housing costs [25]. As such, they are included as a predictive feature in this study.

The combination of these factors makes Stockholm's housing market an interesting case for predictive modeling. Given the complexity of economic and real estate variables, machine learning techniques provide a valuable approach to improving price forecasts.

2.2 Random Forest

Ensemble learning is a method that combines multiple models to improve overall performance. Instead of relying on a single model, it combines the predictions of several models to make a final decision. Random Forest is an ensemble learning method that builds a collection of decision trees and combines their predictions to improve predictive performance. Each tree is trained on a random subset of the data, created through a technique called bagging (bootstrap aggregating). In this process, samples are drawn with replacement from the training set, so some observations may appear multiple times while others are left out. This randomness introduces diversity among the trees, helping to prevent overfitting. By averaging the predictions from many different trees, Random Forest reduces variance and improves the model's ability to generalize [7, 19].

The model can be applied to both classification and regression tasks. In classification problems, the predictions from individual trees are aggregated using a majority vote. In regression, predictions are made by averaging the outputs of each individual decision tree in the forest [19]. The model is designed to combine outputs from several decision trees, thus improving its accuracy.

An important aspect of Random Forest is its ability to handle large datasets with many features, making it effective for tasks such as predicting housing prices, where several dynamically related factors affect the outcome. Furthermore, at each split within a tree, a random subset of features is considered, further decreasing the likelihood of overfitting and increasing model diversity [19].

Random Forest's effectiveness is not only due to its ensemble structure but also due to how well its hyperparameters are tuned. Careful tuning helps manage the complexity of individual trees, promote diversity across the ensemble, and ultimately improve predictive performance [7]. Among the most influential hyperparameters are the number of trees in the forest, controlled by `n_estimators`, and the number of features considered when looking for the best split, specified by `max_features`. The complexity of each tree is further affected by `min_samples_split` and `min_samples_leaf`, which set the minimum number of samples required to split an internal node or to form a leaf. These parameters together influence how deep the trees grow and how well the model generalizes to unseen data [31].

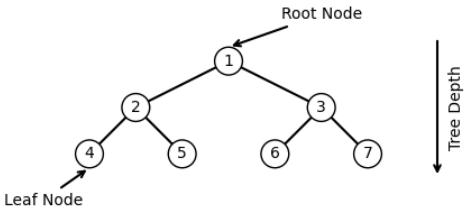


Figure 1: An example of a decision tree.

2.3 Gradient Boosting

Gradient Boosting, like Random Forest, is an ensemble learning method. However, in contrast to Random Forest, Gradient Boosting is designed to improve the predictive performance by combining the strengths of several weak learners, usually in the form of decision trees. The main idea behind Gradient Boosting is to add new models to the ensemble sequentially, rather than building trees independently. Thereby, each new tree aims to correct the errors of the whole ensemble of previous trees. This iterative approach focuses on minimizing a specified loss function, guiding the model toward improved accuracy at each step [29].

The individual trees used in this approach are typically shallow, meaning they have a limited depth, so called stumps. These stumps focus on capturing the most significant patterns in the data without overfitting. Furthermore, Gradient Boosting improves performance by iteratively focusing on the residuals—the differences between the actual and predicted values. Each new tree is trained to correct the errors made by the previous model, allowing the ensemble to gradually reduce prediction error [29].

An important implementation of Gradient Boosting is XGBoost, also known as Extreme Gradient Boosting. It incorporates techniques such as regularization to prevent overfitting, tree pruning to improve generalization, and parallelized computation to speed up training [9].

As with Random Forest, the performance of the Gradient Boosting model is highly sensitive to the choice of hyperparameters. These parameters influence everything from the speed at which the model learns to how well it generalizes without overfitting. Proper tuning is essential to ensure efficient training while maintaining high predictive accuracy [9]. In the case of XGBoost, the learning rate—known as `learning_rate` or `eta`—controls the contribution of each tree to the final prediction, directly impacting how quickly the model adapts to patterns in the data. The number of boosting rounds is set by `n_estimators`, while randomness is introduced through parameters such as `subsample` and `colsample_bytree`, both of which help to reduce overfitting. Additionally, regularization terms such as `reg_lambda` and `reg_alpha` are used to control model complexity and improve generalization. Finally, `min_child_weight` sets the minimum sum of instance weights required in a child node, further influencing the model's ability to avoid overfitting by controlling the growth of trees [31].

2.4 K-Nearest Neighbors

K-Nearest Neighbors, k-NN, is a non-parametric, instance-based learning algorithm commonly used for both classification and regression tasks [11]. In contrast to Random Forest and Gradient Boosting, which are ensemble methods that construct multiple decision trees to identify patterns, k-NN does not build an explicit model. Rather, it stores all training instances and makes predictions by identifying the most similar data points within the dataset [17]. This makes it on one hand very adaptable but on another hand it makes it computationally expensive, especially for larger datasets.

The model's algorithm works by determining the distance between new inputs and all training samples. This can be done using different distance metrics depending on the type of data. A common choice is the *Minkowski* distance, which generalizes multiple distance measures and is defined as:

$$D(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}} \tag{1}$$

where p is a parameter that determines the type of distance metric used [12]. When $p = 2$, the *Minkowski* distance simplifies to the Euclidean distance, given by:

$$D_{\text{Euclidean}}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} \tag{2}$$

For classification tasks, the algorithm identifies the closest neighbor k and assigns the most frequent class among these neighbors. In regression tasks, it predicts the outcome by averaging the values of the nearest neighbors [5].

A critical hyperparameter in k-NN is the choice of k, a smaller k value can potentially lead to overfitting, due to the model becoming sensitive to noise in the data. While larger k values can result in underfitting due to oversmoothing of the predictions. Enhancing accuracy in some cases can be done through employed distance weighting, meaning that closer neighbors have a greater influence on the prediction [11].

One limitation of the model is its sensitivity to the scale of data and its greater dimensionality, meaning that increasing the number of features can decrease the performance. Therefore, data processing steps such as normalization and dimensionality reduction are crucial steps [14]. Despite these challenges, k-NN is valued for its simplicity, interpretability, and effectiveness in scenarios where decision boundaries are irregular and non-linear [11].

2.5 The Dataset

This study uses real estate transaction data from the Booli API [1], a widely used platform for housing market information in Sweden. Booli provides comprehensive details on historical sales, property listings, and price trends. While the API covers transactions across Sweden, this research focuses specifically on properties in Stockholm's inner city.

The dataset contains multiple attributes relevant to housing price prediction, including price, location (address details and geographical coordinates), living area, number of rooms, property type, year

built, and time on market. Spanning from 2012 to 2024, the dataset reflects the full range of transaction data made available through the Booli API. This time frame offers a sufficient basis for identifying market trends and training predictive models. Further details on data retrieval, filtering, and preprocessing are provided in Section 3.

An example of a housing data record retrieved from the Booli API is shown in Table 1. The data in this table is a result of an API call that retrieves properties within a specific location and time range.

Table 1: Example of housing data retrieved from the Booli API

Column	Row
Sale price (SEK)	3 850 000
Address	John Bergs Plan 7
Living area (m^2)	38
Rooms	2
Sale date	2024-12-08
Monthly fee (SEK)	3195
Floor	1
Year built	1937
Property type	Apartment
Area	Kungsholmen
County	Stockholms län
Subarea	Kungsholmen

2.6 Geographical Scope: Stockholm's Inner City

This study focuses exclusively on Stockholm's inner city, which consists of 21 districts grouped into three administrative areas: *Kungsholmens stadsdelsområde*, *Norra innerstadens stadsdelsområde*, and *Södermalms stadsdelsområde*. These include the following areas [34]:

- **Kungsholmen:** Fredhäll, Kristineberg, Kungsholmen, Marieberg, Lilla Essingen, Stadshagen, Stora Essingen.
- **Norra innerstaden:** Djurgården, Hjorthagen, Gärdet, Norra Djurgården, Norrmalm, Skeppsholmen, Beckholmen, Vasastaden, Östermalm.
- **Södermalm:** Gamla stan, Långholmen, Reimersholme, Riddarholmen, Södermalm, Södra Hammarbyhamnen (Hammarby Sjöstad).

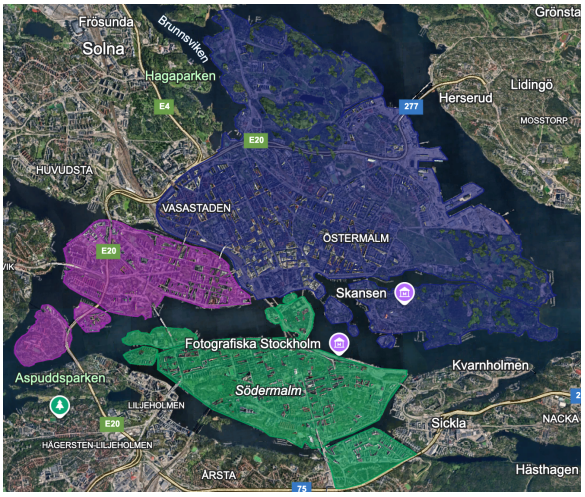


Figure 2: Map of Stockholm's inner city, showing Norra innerstaden (blue), Kungsholmen (purple), and Södermalm (green). The boundaries correspond to the listed subareas and are visualized using Google Maps.

2.7 Related Work

Several studies have been conducted that investigated and compared the effectiveness of predicting housing prices with machine learning models. In Fairfax County, Virginia, machine learning techniques were used to develop predictive models, with findings showing that incorporating variables such as property size, age, and location significantly improved accuracy [30].

Similarly, in Melbourne, Australia, researchers employed machine learning algorithms to predict housing prices. Their results indicated that models accounting for factors such as proximity to amenities, neighborhood characteristics, and economic indicators produced more precise predictions [32].

In Hong Kong, a comparative study evaluated the performance of models such as Support Vector Machines (SVM), Random Forest, and Gradient Boosting. This work demonstrated the value of ensemble and kernel-based approaches in capturing the complex patterns of the housing market [20].

Further research in Metropolitan Adelaide, Australia, incorporated spatio-temporal dependencies into machine learning models. This approach emphasized the importance of both spatial and temporal factors, leading to notable improvements in predictive performance [33].

These studies show that machine learning models effectively predict housing prices, especially when incorporating diverse features such as property characteristics, location, and economic indicators.

3 Method

The methodology of this study was divided into three parts: retrieving and filtering data, preprocessing and feature selection, and model training and evaluation. Existing machine learning models from `scikit-learn` and the `XGBoost` library were used [9, 31]. All implementations were carried out in Python using Google Colab [6].

3.1 Retrieving and Filtering Data

3.1.1 Data Retrieval Real estate transaction data was retrieved from the Booli API. As described in Section 2, the analysis is restricted to apartment sales located in Stockholm's inner city. Geographic filtering was implemented using Booli's area-specific identifiers (`areaId`). The retrieved data was parsed and structured using Pandas for further processing.

3.1.2 Filtering Criteria To improve data quality and relevance, filtering was applied in two stages: during data retrieval and in a subsequent post-processing step.

Filtering at Data Retrieval Stage The Booli API query was restricted using area-specific identifiers to include only apartment listings (`objectType = lägenhet`, Swedish for *apartment*) sold between January 1, 2012, and December 31, 2024.

Post-Retrieval Data Cleaning Listings with non-residential or uncommon property types were excluded, along with outliers such as unusually high prices (above 35 million SEK) or properties located on the 36th floor or higher. Additionally, the lowest 1% of sale prices were removed to avoid unusual or unreliable transactions. To reduce irregularities in location data, free-text subarea names were mapped to standardized district names using a predefined mapping. After initial filtering and removal of rows missing the target variable `Pris` (Swedish for *price*), the dataset was reduced from approximately 120,000 to 88,000 records. Columns with only missing values were also dropped.

3.1.3 Data Processing and Storage The filtered dataset was saved in CSV format. The retained variables include sale price, living area, number of rooms, floor number, monthly fee, year of construction, and sale date. The sale date was transformed into a numerical feature, `Dagar sedan försäljning` (Swedish for *Days since Sale*), representing the number of days since the transaction. This processed dataset was then used for preprocessing and feature selection in Section 3.2.

3.2 Preprocessing and Feature Selection

3.2.1 Splitting the Dataset The dataset was divided based on time to better simulate a real-world forecasting scenario [8, 35]. Transactions from 2012 to 2023 were used to train the models, while data from 2024 was reserved for testing, resulting in a 90:10 split with 78,588 samples in the training set and 8,939 in the test set.

3.2.2 Handling Categorical Variables Categorical variables such as `Bostadstyp` (Swedish for *Type of housing*) and `Område` (Swedish for *Area*) were converted into numerical format using one-hot encoding. The test set was then aligned to match the structure of the encoded training set.

3.2.3 Feature Scaling All numerical features were standardized to ensure they were on a comparable scale. This is particularly important for distance-based models like k-NN, which are sensitive to differences in feature values [16, 22]. Standardization was performed using scikit-learn's `StandardScaler`, which applies the mean and standard deviation from the training data [31]. The same transformation was applied to the test set.

3.2.4 Missing Values Rows missing the target variable `Pris` were removed during the initial filtering stage. Missing values in numerical features—`Boarea` (*living area*), `Antal rum` (*number of rooms*), `Avgift` (*monthly fee*), `Våning` (*floor number*), and `Byggår` (*year built*)—were imputed using the median of each column.

After one-hot encoding and aligning the training and test sets, a second imputation step was applied using scikit-learn's `SimpleImputer` to address any remaining missing values. This ensured the dataset was complete and ready for model training.

3.3 Model Training and Hyperparameter Tuning

The three selected models—Random Forest, XGBoost, and k-NN—were trained and evaluated in two stages. First, model performance was assessed using time-aware cross-validation on the training data from 2012 to 2023. Specifically, `TimeSeriesSplit` was applied to keep the data in time order, making the evaluation more realistic for forecasting applications [8, 31, 35].

In the second stage, each model was fine-tuned using `OptunaSearchCV`, an optimization framework that efficiently searches for high-performing hyperparameter combinations [2]. The search was carried out using 5-fold cross-validation on the training set, again using time-aware folds. Thirty trials were run per model to identify the best hyperparameter configuration. The final optimized models were then evaluated on the separate 2024 test set using standard regression metrics.

3.4 Dataset Characteristics

This section provides an overview of the dataset, including summary statistics and the proportion of missing values.

3.4.1 Descriptive Statistics Table 2 provides descriptive statistics for the dataset after filtering and preprocessing.

Table 2: Descriptive statistics for numerical features

Feature	Mean	Median	Standard Deviation
Sale price (SEK)	5,619,281	4,650,000	3,258,432
Living area (m^2)	59.40	53.00	29.30
Number of rooms	2.29	2.00	1.00
Monthly fee (SEK)	2,743	2,515	1,271
Year built	1933.45	1930	31.74

Figure 3 shows the distribution of apartment prices with a histogram plot, providing a clearer view of price concentration.

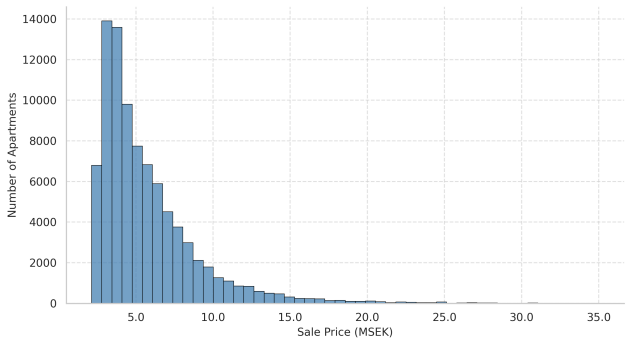


Figure 3: Distribution of apartment prices.

Figure 4 shows annual price per square meter trends by administrative areas, based on the filtered dataset above.



Figure 4: Average price per square meter (SEK) from 2012 to 2024 by administrative areas.

3.4.2 Missing Values Overview As mentioned in Section 3.2, missing data in numerical features such as `Boarea` (*living area*), `Antal rum` (*number of rooms*), and others was handled through imputation using median values. Table 3 provides the proportion of missing values per feature in the original dataset before preprocessing.

Table 3: Proportion of missing values per feature (before preprocessing)

Feature	Missing Data (%)
Year built	10.71
Floor number	10.03
Monthly fee (SEK)	1.71
Living area	0.34
Number of rooms	0.24
Sale price (SEK)	0.00

3.5 Model Evaluation

The final step in this methodology is to assess the predictive performance of the trained models using standard regression error metrics [10]. This study uses four commonly used metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2).

3.5.1 Evaluation Metrics Given a dataset with n observations, let y_i denote the actual price and $\hat{y}_i$ the predicted price for observation i. Let $\bar{y}$ represent the mean of all actual prices.

Mean Absolute Error (MAE) The MAE calculates the average absolute difference between predicted and actual values:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{3}$$

Mean Squared Error (MSE) The MSE calculates the average of squared differences between predicted and actual values:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{4}$$

Root Mean Squared Error (RMSE) The RMSE is the square root of MSE and is expressed in the same unit as the target variable:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{5}$$

Coefficient of Determination (R^2) The R^2 score measures the proportion of variance in the target variable explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{6}$$

4 Results

This section presents the predictive performance of the three machine learning models. Results are shown for both default configurations and after hyperparameter tuning. All metrics are based on a time-based train/test split, with training data from 2012–2023 and testing on 2024.

4.1 Price Trends by Living Area Over Time

Figure 5 shows how the average price per square meter changes with apartment size (living area) for each year between 2012 and 2024. Each line represents a different year, and the line for 2024 is highlighted since this data was used for testing the models. The plot shows a clear trend: smaller apartments generally have a higher price per square meter than larger ones, especially for sizes below 100 m^2. This pattern appears in every year, although the exact price levels and the steepness of the decline vary. In recent years, overall price levels have increased, as seen by the upward shift of the lines.

The figure also suggests that the difference in price per square meter between small and large apartments has changed over time, with some years showing a larger gap than others.

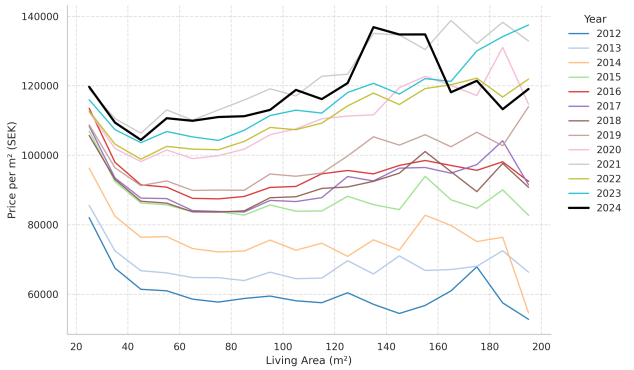


Figure 5: Average price per square meter (SEK) by living area (m^2), segmented by year. The 2024 trend is highlighted in bold.

4.2 Baseline Performance

Model performance was first evaluated using default hyperparameters. A 5-fold time-aware cross-validation was applied on the training set, followed by evaluation on the test data. Results are summarized in Table 4.

Table 4: Baseline performance on 2024 test set (default hyperparameters)

Model	MAE (SEK)	RMSE (SEK)	R^2
Random Forest	748,477	1,242,926	0.90
XGBoost	762,220	1,288,133	0.89
k-NN	870,476	1,386,443	0.87

4.3 Hyperparameter Optimization

Each model was then fine-tuned using `OptunaSearchCV`, with 30 trials and 5-fold time-series cross-validation to maximize the validation R^2. The best configurations were retrained on the full training set and evaluated on the 2024 data. Table 5 presents the resulting performance.

Table 5: Performance on 2024 test set after hyperparameter optimization

Model	MAE (SEK)	RMSE (SEK)	R^2
Random Forest	743,366	1,235,145	0.90
XGBoost	738,989	1,217,465	0.90
k-NN	844,807	1,355,730	0.88

4.4 Best Hyperparameter Configurations

Table 6 shows the hyperparameter values selected during tuning. These configurations correspond to the optimized test results presented in Table 5.

Table 6: Best hyperparameter values for each model

Parameter	Random Forest	XGBoost	k-NN
`n_estimators`	300	300	–
`max_depth`	None	5	–
`min_samples_split`	5	–	–
`min_samples_leaf`	1	–	–
`learning_rate`	–	0.05	–
`subsample`	–	0.8	–
`colsample_bytree`	–	1.0	–
`reg_lambda`	–	1	–
`reg_alpha`	–	1	–
`min_child_weight`	–	3	–
`n_neighbors`	–	–	11
`weights`	–	–	distance
`p`	–	–	2

4.5 Prediction Plots for Optimized Models

To complement the numerical metrics, Figures 6 to 8 visualize the actual versus predicted apartment prices for each optimized model on the 2024 test set.

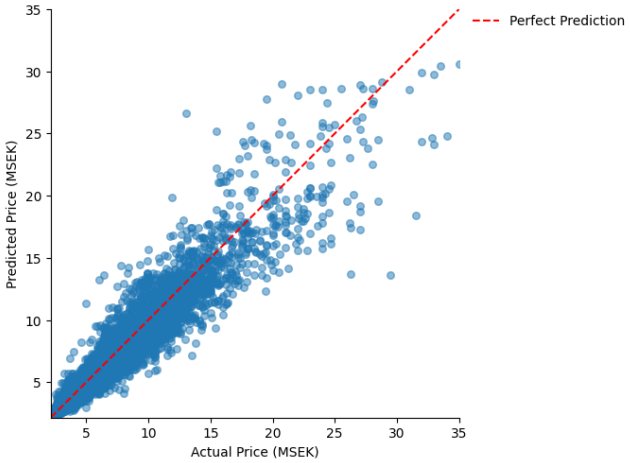


Figure 6: Actual vs. Predicted apartment prices for the optimized Random Forest model. The red dashed line represents perfect prediction.

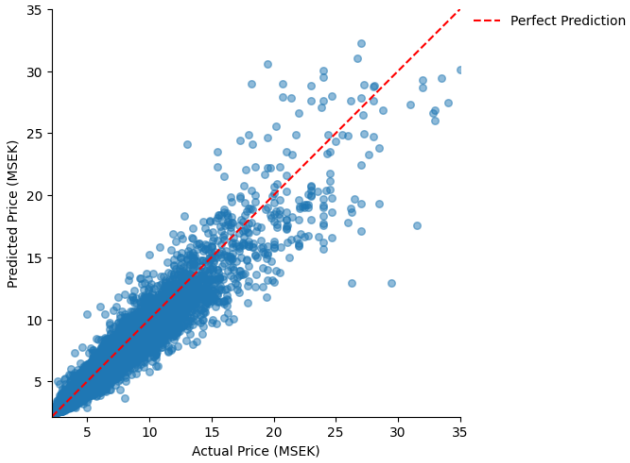


Figure 7: Actual vs. Predicted apartment prices for the optimized XGBoost model. The red dashed line represents perfect prediction.

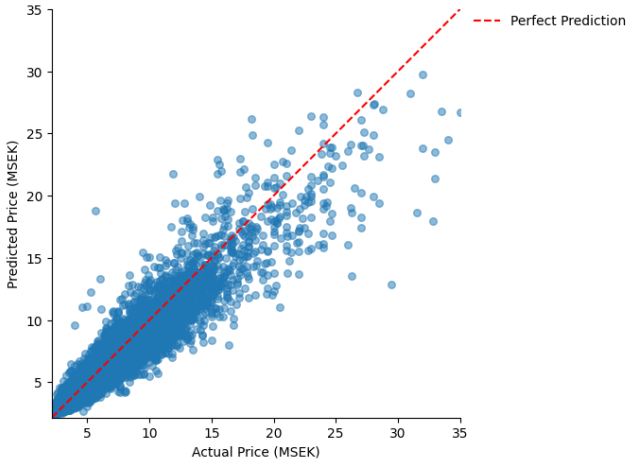


Figure 8: Actual vs. Predicted apartment prices for the optimized k-NN model. The red dashed line represents perfect prediction.

4.5.1 Residual Plots for Optimized Models Residual plots were generated using the 2024 test set. They display prediction errors plotted against actual apartment prices, providing a visual overview of how residuals vary across different price levels.

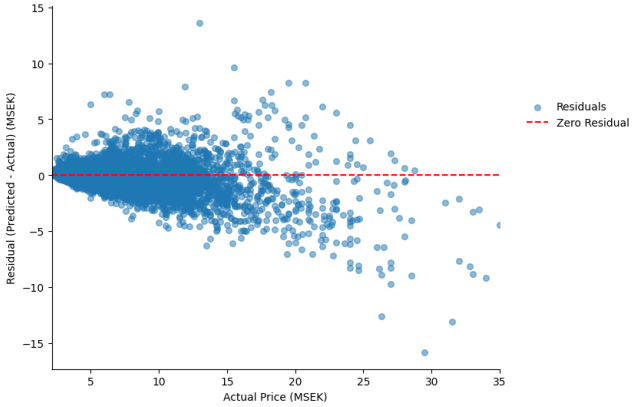


Figure 9: Residuals versus actual prices for the optimized Random Forest model. The red dashed line indicates perfect prediction.

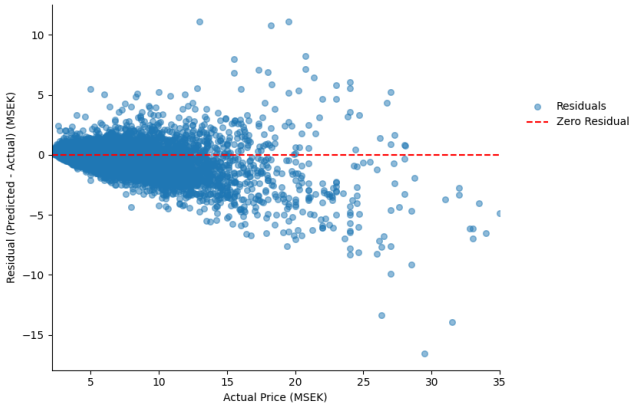


Figure 10: Residuals versus actual prices for the optimized XGBoost model. The red dashed line indicates perfect prediction.

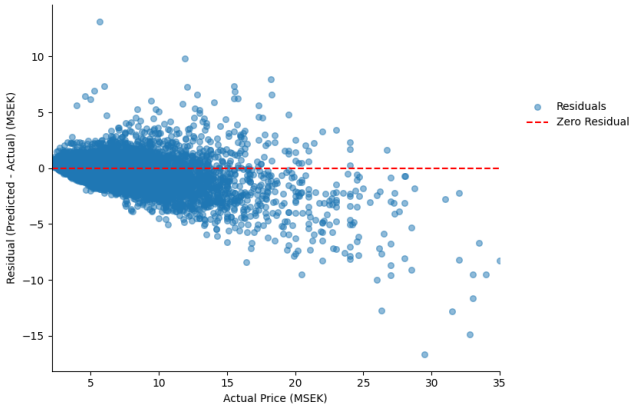


Figure 11: Residuals versus actual prices for the optimized k-NN model. The red dashed line indicates perfect prediction.

4.5.2 Distribution of Residuals Figures 12 to 14 show normalized histograms of residuals for each optimized model. Each histogram is overlaid with a fitted normal distribution curve and a zero-residual reference line, providing a visual summary of the overall error distribution.

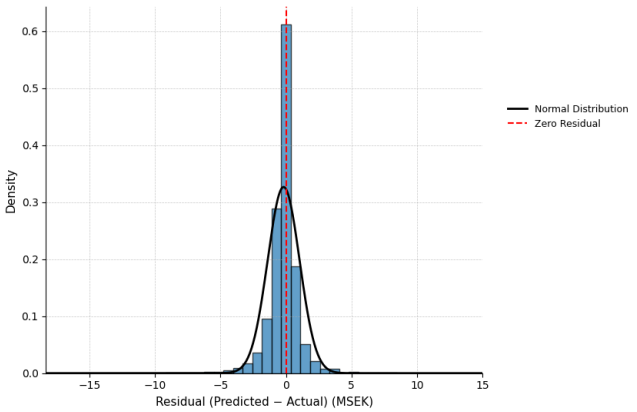


Figure 12: Residual distribution for the optimized Random Forest model. The histogram is normalized and overlaid with a fitted normal distribution and a zero residual line.

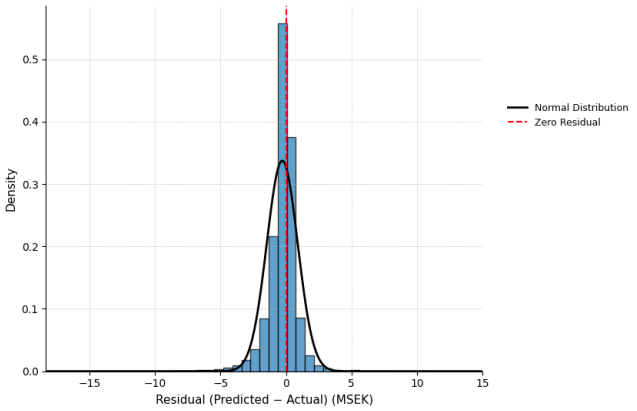


Figure 13: Residual distribution for the optimized XGBoost model. The histogram is normalized and overlaid with a fitted normal distribution and a zero residual line.

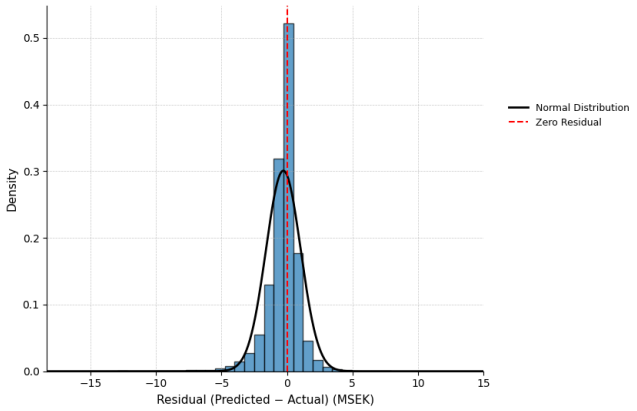


Figure 14: Residual distribution for the optimized k-NN model. The histogram is normalized and overlaid with a fitted normal distribution and a zero residual line.

5 Discussion

The purpose of this study was to evaluate how well three machine learning models—Random Forest, XGBoost, and k-NN—could predict apartment prices in Stockholm's inner city. Specifically, it assessed each model's ability to generalize to unseen data and whether predictive performance could be improved through hyperparameter tuning. Both Random Forest and XGBoost demonstrated high accuracy, with XGBoost achieving the best overall performance based on the lowest RMSE and MAE on the test set.

5.1 Interpretation of Results

All three models demonstrated relatively high predictive accuracy, with R^2 values around 0.90 for Random Forest and XGBoost, and slightly lower for k-NN. After tuning, XGBoost showed the greatest improvement among the models, slightly reducing both MAE and RMSE and achieving a small increase in R^2 compared to the baseline. This suggests that XGBoost is especially sensitive to hyperparameter configuration, aligning with prior research that highlights the model's flexibility and robustness in structured tabular data [9].

The selected evaluation metrics reflect different aspects of predictive performance. MAE indicates the average size of prediction errors, treating all deviations equally. MSE places greater emphasis on larger errors due to the squaring of differences, which increases sensitivity to significant deviations. RMSE, as the square root of MSE, expresses the average error in the same unit as the target variable, improving interpretability. The R^2 score represents the proportion of variance in the target variable that is accounted for by the model, with a value of 1 indicating perfect predictions and 0 implying no improvement over simply predicting the mean.

The residual plots showed that prediction errors tended to be larger for higher-priced apartments, especially with Random Forest and k-NN. This pattern likely stems from heteroscedasticity, a statistical principle often encountered in machine learning and regression models. It refers to a situation where the variability of prediction errors changes across different subsets of the data [38]. In this case, the data imbalance, where higher-priced properties are

less frequent, causes more variability in error for these transactions. As a result, the models have more difficulty making accurate predictions in these cases. Models like XGBoost, Random Forest, and k-NN, which depend on local data density, tend to struggle in more sparse regions, leading to weaker generalization. Similar trends have been observed in other real estate studies, where performance typically drops in underrepresented price segments due to higher uncertainty [11, 18, 27].

In addition, the residual histograms show that all three models make prediction errors that are mostly centered around zero and roughly symmetric, meaning the models generally do not overpredict or underpredict prices in a consistent way. However, the errors are not perfectly normally distributed, especially for Random Forest and k-NN, where there are more large errors. The XGBoost model has a tighter and more balanced error distribution, which suggests that it predicts prices more consistently than the other models.

5.2 Relation to Previous Work

These results are consistent with earlier literature, identifying Random Forest and boosting algorithms as strong performers in housing price prediction tasks [20, 30, 32]. The fact that XGBoost slightly outperformed Random Forest supports conclusions drawn in other contexts, where gradient boosting has shown better bias-variance trade-offs [32, 33]. The k-NN model, while intuitive and easy to interpret, lagged behind in both baseline and optimized scenarios, reflecting its sensitivity to feature scaling and the distribution of nearby training examples [20].

5.3 Methodological Considerations

The use of a time-based data split better simulates a realistic forecasting environment compared to random sampling. Likewise, time-aware cross-validation made sure that no future data influenced model training, thus avoiding data leakage. These methodological choices make the results more applicable to real-world scenarios and follow recommended practices for time-based forecasting [8].

Using `OptunaSearchCV` made it possible to test different hyperparameter combinations efficiently without requiring too much computing power. However, because the number of trials was limited, the best possible parameter values may not have been found. Increasing the number of trials or exploring more advanced search techniques, such as Bayesian optimization, could potentially lead to better results [4].

5.4 Practical and Societal Implications

Accurate models for predicting apartment prices can be useful for real estate agents, home buyers, and policymakers. Models like Random Forest and XGBoost could help with setting property prices, making investment decisions, and analyzing market trends. On a broader level, better price predictions might make the housing market more transparent, helping to reduce the information gap between buyers and sellers. However, since the models are based on historical sales data, there is a risk that they might reinforce existing inequalities if certain areas or types of properties are underrepresented in the data.

5.5 Limitations and Future Work

One limitation of this study is that it does not include external factors such as interest rates, economic trends, or how close an apartment is to public transport—all of which can affect housing prices. Adding these types of contextual variables could help the models make more accurate predictions and make it easier to understand what influences the results. Given that past research has shown how strongly economic indicators and interest rates impact housing markets, it is possible that including these factors could have improved the accuracy of the predictive models [13, 28].

In addition, no interviews or consultations were conducted with real estate professionals to identify which factors they consider most important when pricing apartments. Getting input from experts could help choose relevant features, check whether the predictions make sense in practice, and point out important factors that are missing from the data. This is a limitation of the current study and could be addressed in future work to better connect the models to real-world knowledge.

Furthermore, while residual plots and error metrics help evaluate model performance, using feature importance techniques could provide deeper insights into which factors have the greatest impact on predicted prices. One technique that could be used in future work is SHAP (SHapley Additive exPlanations), which assigns importance scores to features based on their contribution to model predictions. This could help identify key factors influencing prices and make the models easier to interpret [26].

Another potential approach is to segment the dataset into price ranges—such as below 5 million SEK, 5-10 million SEK, and above 10 million SEK—and train separate models for each segment. This could help reduce prediction errors in higher-priced segments, where the residual plots suggest that current models struggle with variability. Price-specific models could help capture the unique patterns and factors that affect apartments in each price range.

Future research could build on this study by testing more advanced models, such as neural networks or attention-based methods [36, 37]. It could also be helpful to include more detailed geographic information—like the exact location of each apartment or how close it is to services and amenities, the floor number, or whether the apartment has a balcony—to improve prediction accuracy [23, 32]. Finally, testing the models on future sales data that was not part of this study would give a better sense of how well they perform over time [3].

5.6 Contributions

This study contributes to the field of real estate price prediction by comparing the performance of different machine learning models in a time-aware forecasting context. The results show that, with careful data preparation and hyperparameter tuning, tree-based models—particularly XGBoost—can provide quite accurate price estimates in urban housing markets. The approach used in this study can be applied to other cities as well, assuming that comparable structured sales data is available.

6 Conclusion

This study set out to evaluate how well three different machine learning models—Random Forest, XGBoost, and k-Nearest Neighbors—can predict apartment prices in Stockholm's inner city. The models were tested using a time-based data split, with training data from 2012 to 2023 and testing on unseen data from 2024.

The results showed that both Random Forest and XGBoost achieved strong predictive performance, with XGBoost performing best overall after hyperparameter tuning. In contrast, the k-NN model showed lower accuracy, particularly in price ranges with fewer data points. The study also found that prediction errors were larger for high-end apartments, which is likely due to data imbalance in those price segments.

These findings are consistent with previous research that highlights the effectiveness of tree-based models for housing price prediction. They also show the value of using time-aware validation and hyperparameter tuning when working with real-world data.

The methods and results in this thesis can be used in other urban housing markets with similar data. Future work could include additional variables such as interest rates or location-based features, or explore more advanced models like neural networks to improve prediction accuracy even further.

7 References

[1] 2024. Booli - Real Estate Data in Sweden. https://www.booli.se/ Accessed: 2024-03-12.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

[3] Mehmet Balcilar, Rangan Gupta, and Stephen M Miller. 2015. The out-of-sample forecasting performance of nonlinear models of regional housing prices in the US. *Applied Economics* 47, 22 (2015), 2259–2277.

[4] James Bergstra, Daniel Yamins, and David Cox. 2013. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. In *International conference on machine learning*. PMLR, 115–123.

[5] Christopher M. Bishop. 2006. *Pattern Recognition and Machine Learning*. Springer, New York. https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf

[6] Ekaba Bisong. 2019. Google colaboratory. In *Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners*. Springer, 59–64.

[7] Leo Breiman. 2001. Random forests. *Machine learning* 45 (2001), 5–32.

[8] Vitor Cerqueira, Luis Torgo, and Igor Mozetič. 2020. Evaluating time series forecasting models: An empirical study on performance estimation methods. *Machine Learning* 109, 11 (2020), 1997–2028.

[9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) *(KDD '16)*. ACM, New York, NY, USA, 785–794. https://doi.org/10.1145/2939672.2939785

[10] Davide Chicco, Matthijs J Warrens, and Giuseppe Jurman. 2021. The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation. *Peerj computer science* 7 (2021), e623.

[11] Thomas Cover and Peter Hart. 1967. Nearest neighbor pattern classification. *IEEE transactions on information theory* 13, 1 (1967), 21–27.

[12] Felipe SLG Duarte, Ricardo A Rios, Eduardo R Hruschka, and Rodrigo F de Mello. 2019. Decomposing time series into deterministic and stochastic influences: A survey. *Digital Signal Processing* 95 (2019), 102582.

[13] John V Duca, John Muellbauer, and Anthony Murphy. 2011. House prices and credit constraints: Making sense of the US experience. *The economic journal* 121, 552 (2011), 533–551.

[14] Sahibsingh A Dudani. 1976. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics* 4 (1976), 325–327.

[15] Marko Grebovic, Luka Filipovic, Ivana Katnic, Milica Vukotic, and Tomo Popovic. 2023. Machine learning models for statistical analysis. *Int. Arab J. Inf. Technol.* 20, 3A (2023), 505–514.

[16] Jiawei Han, Jian Pei, and Hanghang Tong. 2022. *Data mining: concepts and techniques*. Morgan kaufmann.

[17] Trevor Hastie, Robert Tibshirani, Jerome H Friedman, and Jerome H Friedman. 2009. *The elements of statistical learning: data mining, inference, and prediction*. Vol. 2. Springer.

[18] Haibo He and Yunqian Ma. 2013. Imbalanced learning: foundations, algorithms, and applications. (2013).

[19] Tin Kam Ho. 1995. Random decision forests. In *Proceedings of 3rd international conference on document analysis and recognition*, Vol. 1. IEEE, 278–282.

[20] Winky KO Ho, Bo-Sin Tang, and Siu Wai Wong. 2021. Predicting property prices with machine learning algorithms. *Journal of Property Research* 38, 1 (2021), 48–70.

[21] Gustav Ingman. 2022. *Essays on Stockholm's real estate market 1730–2020*. Ph. D. Dissertation. Department of Economic History and International Relations, Stockholm University.

[22] Anil K Jain, Robert P. W. Duin, and Jianchang Mao. 2000. Statistical pattern recognition: A review. *IEEE Transactions on pattern analysis and machine intelligence* 22, 1 (2000), 4–37.

[23] Yuhao Kang, Fan Zhang, Wenzhe Peng, Song Gao, Jinmeng Rao, Fabio Duarte, and Carlo Ratti. 2021. Understanding house price appreciation using multi-source big geo-data and machine learning. *Land use policy* 111 (2021), 104919.

[24] Jun Hyeok Lee and Dae Ryong Kang. 2020. From Traditional Statistical Methods to Machine and Deep Learning for Prediction Models. *Cardiovascular Prevention and Pharmacotherapy* 2, 2 (2020), 50–55.

[25] Anna Lidberg. 2018. The finances of housing cooperatives and financial stability. *Economic Commentaries of the Sverige Bank* 4 (2018).

[26] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017).

[27] Lucas Mentch and Giles Hooker. 2016. Quantifying uncertainty in random forests via confidence intervals and hypothesis tests. *Journal of Machine Learning Research* 17, 26 (2016), 1–41.

[28] John Muellbauer and Anthony Murphy. 1997. Booms and busts in the UK housing market. *The Economic Journal* 107, 445 (1997), 1701–1727.

[29] Alexey Natekin and Alois Knoll. 2013. Gradient boosting machines, a tutorial. *Frontiers in neurorobotics* 7 (2013), 21.

[30] Byeonghwa Park and Jae Kwon Bae. 2015. Using machine learning algorithms for housing price prediction: The case of Fairfax County, Virginia housing data. *Expert systems with applications* 42, 6 (2015), 2928–2934.

[31] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[32] The Danh Phan. 2018. Housing price prediction using machine learning algorithms: The case of Melbourne city, Australia. In *2018 International conference on machine learning and data engineering (iCMLDE)*. IEEE, 35–42.

[33] Ali Soltani, Mohammad Heydari, Fatemeh Aghaei, and Christopher James Pettit. 2022. Housing price prediction incorporating spatio-temporal dependency into machine learning algorithms. *Cities* 131 (2022), 103941.

[34] Stockholms stad. 2025. Områdesfakta – statistik om stadens delområden. https://start.stockholm/om-stockholms-stad/utredningar-statistik-och-fakta/statistik/omradesfakta/ Accessed: 2025-04-08.

[35] Leonard J Tashman. 2000. Out-of-sample tests of forecasting accuracy: an analysis and review. *International journal of forecasting* 16, 4 (2000), 437–450.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[37] Pei-Ying Wang, Chiao-Ting Chen, Jain-Wun Su, Ting-Yun Wang, and Szu-Hao Huang. 2021. Deep learning model for house price prediction using heterogeneous data analysis along with joint self-attention mechanism. *IEEE access* 9 (2021), 55244–55259.

[38] Halbert White. 1980. A heteroskedasticity-consistent covariance matrix estimator and a direct test for heteroskedasticity. *Econometrica: journal of the Econometric Society* (1980), 817–838.